

Module : 4

Dynamic Programming Back Tracking &

Branch and Bound.

→ Dynamic Programming :-

- Used in optimization problems.
- It is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions.
- Combining the solutions of subproblems.
- It solves each subproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time.
- Two main properties of a problem that a problem must have in order for Dynamic programming to be applicable.

① Optimal substructure.

② Overlapping subproblem.

→ Optimal Substructure :-

- If the optimal solution of the given problem can be obtained using optimal solutions of its sub-problems.

E.g: If x lies in the shortest path from a source node u to destination node v , then the shortest path from u to v is the combination of shortest path from u to x and x to v .

→ Overlapping sub-problems:

- It also combines solutions to sub-problems.
- It mainly used where the solution of one sub-problem is needed repeatedly.
- computed solutions are stored in a table.

→ Steps of Dynamic Programming Approach:

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution typically in a bottom-up fashion.
4. Construct an optimal solution from the computed information.

→ Applications of D.P Approach:

1. Matrix chain multiplication
2. Longest common subsequence.
3. TSP.

→ Principle of Optimality:

It states that an optimal sequence of decisions has the property that whatever the initial state & decision are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.

→ Matrix chain multiplication:

- Problem is stated as follows:

"Given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$ matrix A_i has dimension $P_{i-1} \times P_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in a way that minimizes the no. of scalar multiplications."

Note:

- Two matrices A & B can be multiplied only if the no. of columns of $A =$ no. of rows of B .

- If A is $p \times q$ matrix and B is $q \times r$ matrix, then the resulting matrix C is a $p \times r$ matrix.
- The time to compute C is denoted by the number of scalar multiplications which is pqr .

Example:

- Consider the problem of a chain $\langle A_1, A_2, A_3 \rangle$ of three matrices. For example, suppose A_1 is a 10×100 matrix, A_2 is a 100×5 matrix and A_3 is a 5×50 matrix. we can fully parenthesize the product $A_1 A_2 A_3$ in two distinct ways: $(A_1 A_2) A_3$ & $A_1 (A_2 A_3)$
- $\Rightarrow$ computing $(A_1 A_2) A_3$ needs $(10 \times 100 \times 5) + (10 \times 5 \times 50)$
 10×5
 $5000 + 2500 = 7500$

$$A_1 (A_2 A_3) = (100 \times 5 \times 50) + (10 \times 100 \times 50) = 25000 + 50000 = 75000$$

1st method is 10 times faster and hence more efficient.

Algorithm 1 -

- table s is used to construct an optimal solution.
- table m is used to store $m[i, j]$ costs.
- The m table uses only the main diagonal & upper triangle and s table uses only upper Δ

MATRIX-CHAIN-ORDER(P)

$n = P.length - 1$

let $m[1 \dots n, 1 \dots n]$ and $s[1 \dots n-1, 2 \dots n]$ be new tables

for $i = 1$ to n

$m[i, i] = 0$

for $l = 2$ to n

for $i = 1$ to $n - l + 1$

$j = i + l - 1$

$m[i, j] = \infty$

for $k = i$ to $j - 1$

$q = m[i, k] + m[k+1, j] + P_k \cdot P_j$

if $q < m[i, j]$

$m[i, j] = q$

$s[i, j] = k$

PRINT - OPTIMAL - PARENS (s, i, j)

if $i = j$

print " A_i "

else print "("

PRINT - OPTIMAL - PARENS ($s, i, s[i, j]$)

PRINT - OPTIMAL - PARENS ($s, s[i, j] + 1, j$)

print ")"

Analysis!

3 nested loops & each loop index (1, 1, 1) takes on atmost $n-1$ values i.e each loop has $O(n)$ iterations.

So this algorithm has running time $O(n^3)$.

It requires $O(n^2)$ space to store the matrices tables.

? Given a chain of 4 matrices $\langle A_1 A_2 A_3 A_4 \rangle$ with dimensions $\langle 5 \times 4 \rangle \langle 4 \times 6 \rangle \langle 6 \times 2 \rangle \langle 2 \times 7 \rangle$ respectively

Using D.P find the minimum number of scalar multiplications needed & also write optimal multiplication order.

$\Rightarrow$

A_1	A_2	A_3	A_4
5×4	4×6	6×2	2×7

	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

m table

	1	2	3
1			
2			
3			

s table

$$m[1,1] = m[2,2] = m[3,3] = m[4,4] = 0.$$

$$m[1,2] = 5 \times 4 \times 6 = 120$$

$$m[2,3] = 4 \times 6 \times 2 = 48$$

$$m[3,4] = 6 \times 2 \times 7 = 84$$

$$m[1,3] = \min \{ m[1,1] + m[2,3] + 5 \times 4 \times 3, m[1,2] + m[3,3] + 5 \times 6 \times 2 \}$$

$$= \min \{ 0 + 48 + 40, 120 + 0 + 60 \}$$

$$= \min \{ 88, 180 \}$$

$$= 88$$

$$m[2,4] = \min \{ m[2,2] + m[3,4] + 4 \times 6 \times 7, m[2,3] + m[4,4] + 4 \times 2 \times 7 \}$$

$$= \min \{ 0 + 84 + 168, 48 + 0 + 56 \}$$

$$= \min \{ 252, 104 \} = 104$$

$$\begin{aligned}
 m[1,4] &= \min \{ m[1,1] + m[2,4] + 5 \times 4 \times 7, \\
 &\quad m[1,2] + m[3,4] + 5 \times 6 \times 7, m[1,3] + \\
 &\quad m[4,4] + 5 \times 2 \times 7 \} \\
 &= \min \{ 0 + 104 + 140, 120 + 84 + 210, \\
 &\quad 88 + 0 + 70 \} \\
 &= \min \{ 244, 414, 158 \} \\
 &= \underline{\underline{158}}
 \end{aligned}$$

Optimal multiplication order

$$((A_1 (A_2 A_3)) A_4)$$

→ All Pairs Shortest Path Problem

→ Floyd - Warshall Algorithm: —

- The all pair shortest path algorithm is known as Floyd-Warshall algorithm.
- Find shortest path between all pairs of vertices, negative edges are allowed.
- Algorithm will generate matrix, which will represent the minimum distance from any node to all others nodes in the graph.

Algorithm: —

Let $v =$ no. of vertices in graph.

Let $dist = v \times v$ array of minimum distances.

for each vertex v

$dist[v][v] = 0$

for each edge (u, v)

$dist[u][v] = weight(u, v)$

for k from 1 to v

for i from 1 to v

for j from 1 to v

if $(dist[i, j] > dist[i, k] + dist[k, j])$

$dist[i, j] = dist[i, k] + dist[k, j]$

end if.

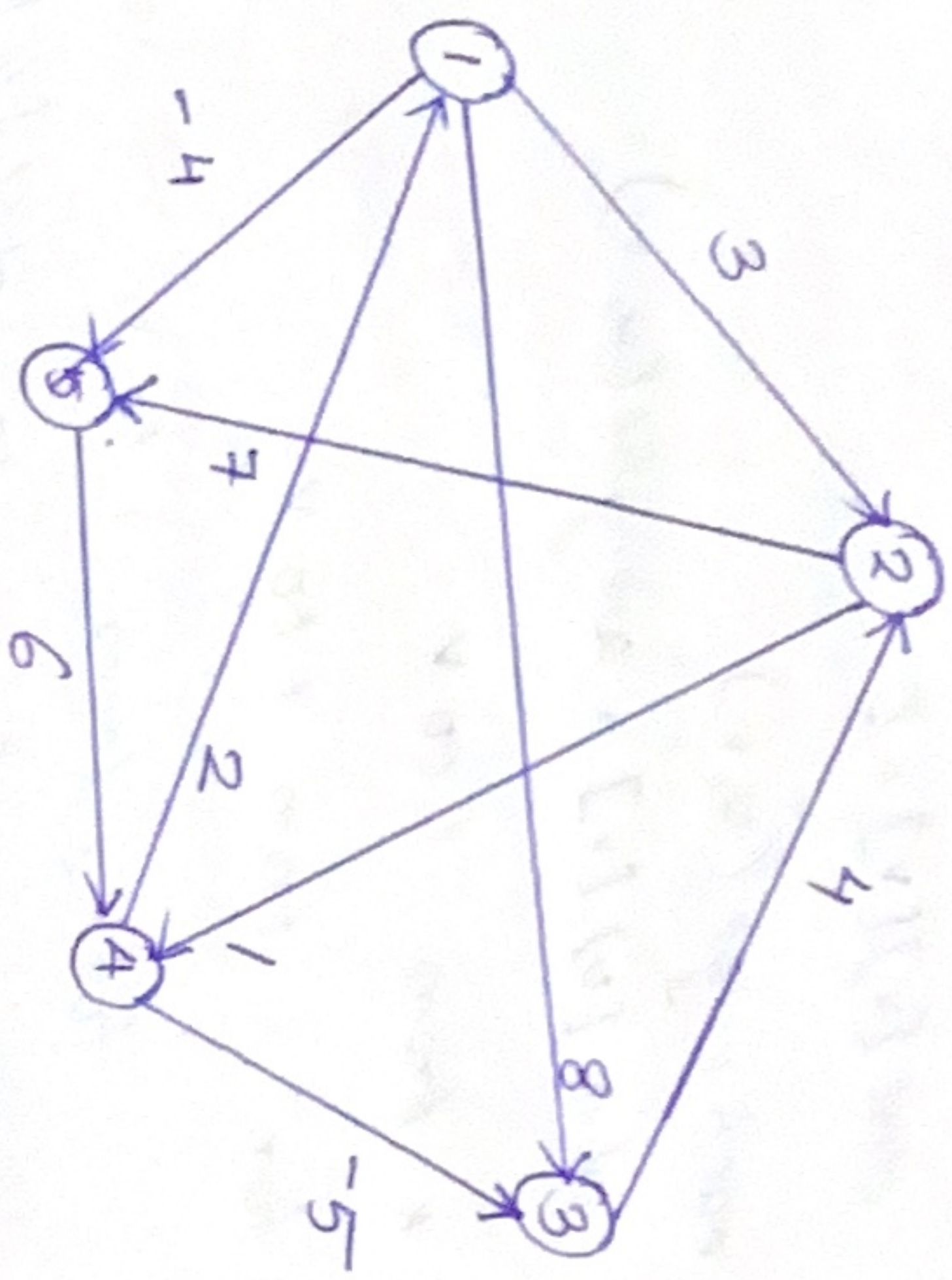
Analysis: —

3 loops → $O(n^3)$.

$n \rightarrow$ no. of nodes in the given graph.

Space complexity $O(n^2)$

Consider the following directed acyclic graph. Using Floyd Warshall algorithm, find the shortest path distance between every pair of vertices.



Step 1:

- Remove all the self loops and parallel edges from the graph.
- In the given graph, there are neither self edges nor parallel edges.

Step 2:

Write initial distance matrix:

$$D^{(0)} = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & -4 & \infty \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & -5 & \infty \\ 2 & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

After $k=1$

$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

$$d_{14}^{(1)} = \min(d_{14}^{(0)}, d_{15}^{(0)} + d_{54}^{(0)}) = \infty$$

$$= \min(\infty, 0 + \infty) = \infty$$

$$d_{15}^{(1)} = \min(d_{15}^{(0)}, 0 + (-4)) = -4$$

$$d_{21}^{(1)} = \min(\infty, \infty + 0) = \infty$$

$$d_{23}^{(1)} = \min(\infty, \infty + 8) = \infty$$

$$d_{31}^{(1)} = \min(\infty, \infty + 0) = \infty$$

$$d_{35}^{(1)} = \min(\infty, \infty + (-4)) = \infty$$

$$d_{42}^{(1)} = \min(\infty, 2 + 3) = 5$$

$$d_{43}^{(1)} = \min(\infty, 2 + 8) = 10$$

$$d_{45}^{(1)} = \min(\infty, 2 + (-4)) = -2$$

$$d_{51}^{(1)} = \min(\infty, \infty + 0) = \infty$$

$$D_{ij}^{(1)} = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & -5 & \infty \\ 2 & 5 & 10 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix} \end{matrix}$$

Aben $k=2$

$$d_{14}^{(2)} = \min(\infty, 3+1) = 4$$

$$d_{21}^{(2)} = \min(\infty, 0+\infty) = \infty$$

$$d_{34}^{(2)} = \min(-5, 4+1) = -5$$

$$d_{85}^{(2)} = \min(\infty, 4+7) = 11$$

$$d_{43}^{(2)} = \min(10, 5+\infty) = 10$$

$$D_{ij}^{(2)} = \begin{matrix} & 0 & 3 & 8 & 4 & -4 \\ \begin{matrix} 0 \\ \infty \end{matrix} & & 0 & \infty & 1 & 7 \end{matrix}$$

$$\begin{matrix} & 2 & 5 & 10 & 0 & -2 \\ \begin{matrix} \infty \\ \infty \end{matrix} & & \infty & \infty & 6 & 0 \end{matrix}$$

Aben $k=3$

$$d_{14}^{(3)} = \min(4, 8+(-5)) = 3$$

$$D_{ij}^{(3)} = \begin{matrix} & 0 & 3 & 8 & 3 & -4 \\ \begin{matrix} \infty \\ \infty \end{matrix} & & 0 & \infty & 1 & 7 \end{matrix}$$

$$\begin{matrix} & 2 & 5 & 10 & 0 & -2 \\ \begin{matrix} \infty \\ \infty \end{matrix} & & \infty & \infty & 6 & 0 \end{matrix}$$

Aben $k=4$

$$d_{21}^{(4)} = \min(\infty, 1+2) = 3$$

$$d_{35}^{(4)} = \min(\infty, 1+10) = 11$$

$$d_{35}^{(4)} = \min(7, 1+(-2)) = -1$$

$$d_{31}^{(4)} = \min(\infty, -5+2) = 3$$

$$d_{32}^{(4)} = \min(4, -5+5) = 0$$

$$d_{51}^{(4)} = \min(\infty, 6+2) = 8$$

$$d_{52}^{(4)} = \min(\infty, 6+5) = 11$$

$$d_{53}^{(4)} = \min(\infty, 6+10) = 16$$

$$D_{ij}^{(4)} = \begin{matrix} & 0 & 3 & 8 & 3 & -4 \\ \begin{matrix} 3 \\ -3 \end{matrix} & & 0 & 11 & 1 & -1 \end{matrix}$$

$$\begin{matrix} & 2 & 5 & 10 & 0 & -2 \\ \begin{matrix} 8 \\ 2 \end{matrix} & & 11 & 16 & 6 & 0 \end{matrix}$$

$k=5$

$$D_{ij}^{(5)} = \begin{matrix} & 0 & 3 & 8 & 3 & -4 \\ \begin{matrix} 3 \\ -3 \end{matrix} & & 0 & 11 & 1 & -1 \end{matrix}$$

$$\begin{matrix} & 2 & 5 & 10 & 0 & -2 \\ \begin{matrix} 8 \\ 2 \end{matrix} & & 11 & 16 & 6 & 0 \end{matrix}$$

→ BACKTRACKING

→ Control abstraction: -

Algorithm Backtrack(k)

```

{
  for (each  $x[k] \in T(x[1], \dots, x[k-1])$ )
  {
    if ( $B_k(x[1], x[2], \dots, x[k] \neq 0)$ ) then
    {
      if ( $x[1], x[2], \dots, x[k]$  is a path
      to an answer node)
      then write ( $x[1:k]$ )
      then Backtrack( $k+1$ )
    }
  }
}

```

$x[1:n]$: solution vector.

$T()$ → generating function

$B()$ → Bounding function

- Generating function $T(x_1, x_2, \dots, x_{k-1})$ be the

set of all possible values for x_k such that

$(x, x_2, \dots, x_k)$ is also a path to a problem state.

$T(x_1, x_2, \dots, x_k) = \phi$.

- Bounding function $B_k(x_1, x_2, \dots, x_k)$ is false for a path $(x_1, x_2, \dots, x_k)$ from the root node to a problem state, then the path cannot be extended to reach an answer node.

- Recursive version is initially invoked by Backtrack(1).

→ N-Queens Problem: -

- Problem is to place n -queens in such a manner on an $n \times n$ chessboard that no queens attack each other by being in the same row, column or diagonal.

- $n=1$, the problem has a trivial solution & no solution exists for $n=2, n=3$.

- We consider 4 queens problem & then generate it to n -queens problems.

- Each queen must be on different row.

		q ₁	
q ₂			
			q ₃
	q ₄		

1. Explicit constraint.

2. implicit constraint

→ 1.) Explicit constraint: —

- Rules that restrict each x_i to take values only from a given set.

e.g: There are n columns 1:0

$$\{1, 2, 3, \dots, 5\}$$

50c 501n space contains:

So in space $\dots \times |S_n| = n \times n \times \dots \times n$
 $|S_1| \times |S_2| \times |S_3| \times \dots = n^n$ tuples

→ 2. implicit constraint! —

→ d. improve

- Rules that determine which of the tuples in the solution space of 1 satisfy the criterion function.

→ Algorithm: —
place (k, i)

```

    }
    for j ← 1 to k-1
    do if (x[j] = i)
        return x[j] - i = (abs(j-k))

```

[illegible]

- During state space tree generation E -node remains E -node until it is dead

- Also strategies:

- Breadth First Search (BFS): -

- FIFO live nodes are placed in a queue

- Depth Search (D-Search)

- LIFO. live nodes are placed in a stack

- Least Cost Search (LC Search)

- To improve searching speed, we can use a ranking function $C(.)$

Algorithm LC Search (t) $\rightarrow$ node

{ if t is answer then return it get parent as t

if t is an answer node then output t and return.

$E = t$

with the list of live nodes to be empty

repeat

{ for each child x of E do (above all child E)

{ if x is an answer node then output the path from x to t & return

x is answer node display path from x to t and exit

either queue / $\leftarrow$ Add (x) $\rightarrow$ not answer node then add it to live node list

stack $x \rightarrow$ Parent = E . and set x as parent as E

} if there are no more live nodes then if live node list is empty then

{ write "no answer node" no answer node

return

} all nodes from live nodes and take by using ranking fn find the

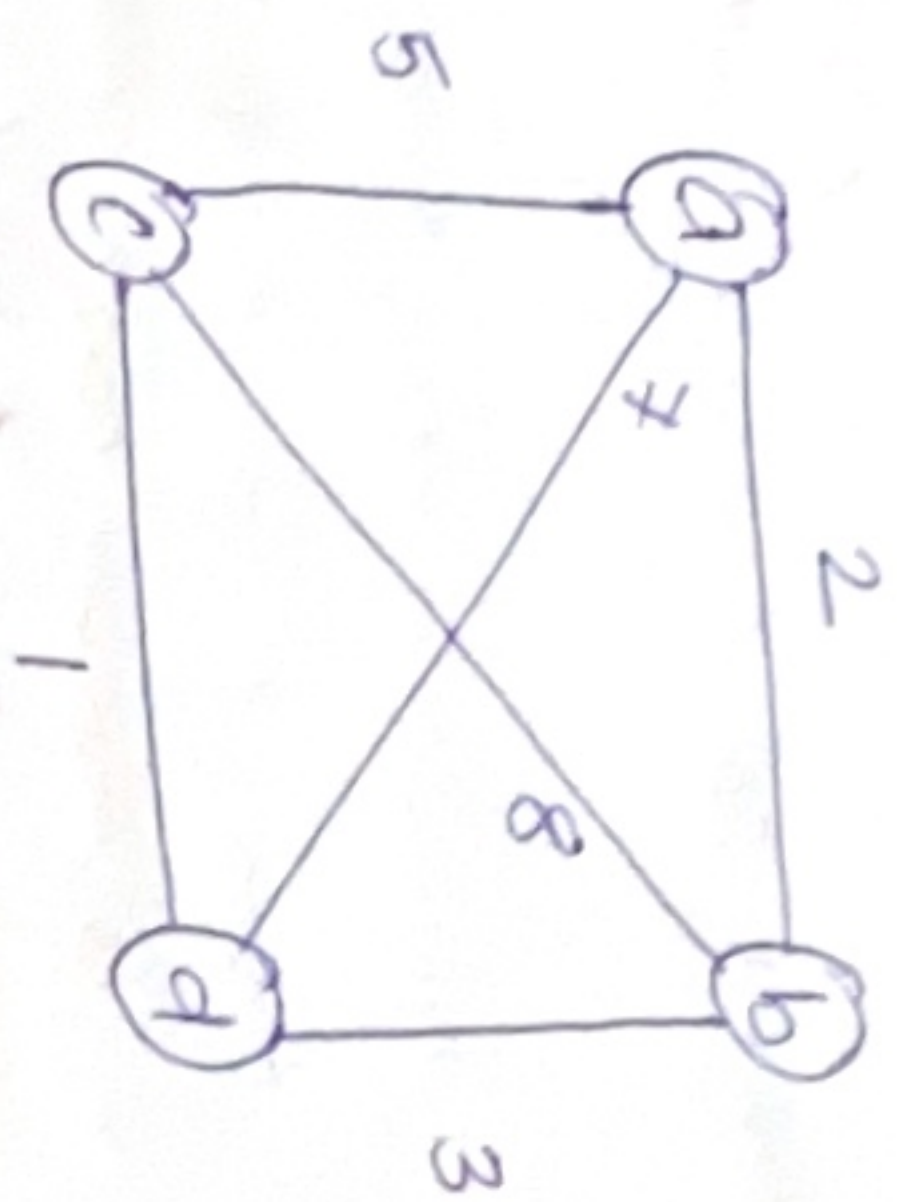
$E = \text{least } C(.)$ cost and which node having least cost will extract &

} until (false) set it as parent E -node

}

$\rightarrow$ TSP using Branch & Bound! -

Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible tour that visits every city exactly once & returns to the starting point.



Adjacency matrix:

	a	b	c	d
a	∞	2	5	7
b	2	∞	8	∞
c	5	8	∞	1
d	7	3	1	∞

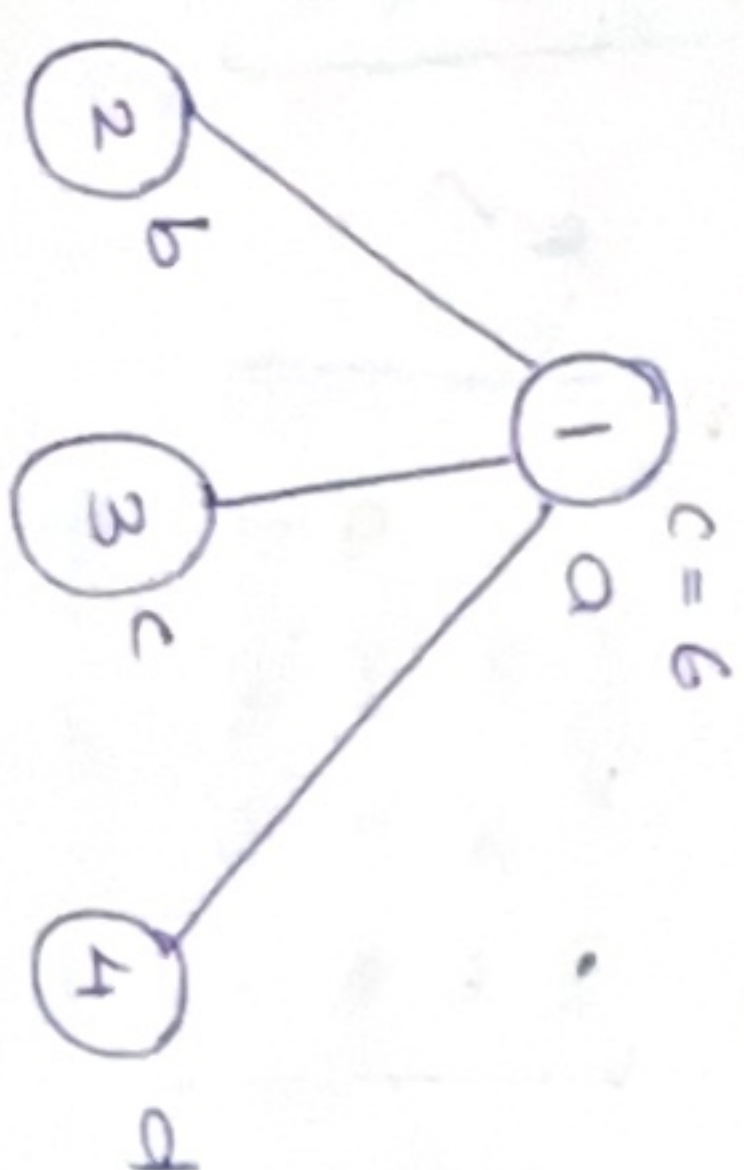
Perform row & column reduction.

$$\begin{bmatrix} \infty & 2 & 5 & 7 \\ 2 & \infty & 8 & 3 \\ 5 & 8 & \infty & 1 \\ 7 & 3 & 1 & \infty \end{bmatrix} \xrightarrow{R.R.} \begin{bmatrix} \infty & 0 & 3 & 5 \\ 0 & \infty & 6 & 1 \\ 4 & 1 & \infty & 0 \\ 6 & 2 & 0 & \infty \end{bmatrix} \xrightarrow{C.R.} \begin{bmatrix} \infty & 0 & 3 & 5 \\ 0 & \infty & 6 & 1 \\ 4 & 1 & \infty & 0 \\ 6 & 2 & 0 & \infty \end{bmatrix} M_1$$

Total reduction = 6

- M_1 is the matrix for node 1.

- Node 1 is K-node. Generate child nodes.



- Find matrix & cost of node 2.

- Set row a & column b elements are ∞ .

- Set $M_1[b,a] = \infty$

$$\begin{bmatrix} \infty & \infty & \infty & 0 \\ \infty & \infty & 6 & 1 \\ 4 & \infty & \infty & 0 \\ 6 & \infty & 0 & \infty \end{bmatrix} \xrightarrow{R.R.} \begin{bmatrix} \infty & \infty & \infty & 0 \\ \infty & \infty & 5 & 0 \\ 4 & \infty & \infty & 0 \\ 6 & \infty & 0 & \infty \end{bmatrix} \xrightarrow{C.R.} \begin{bmatrix} \infty & \infty & \infty & 0 \\ \infty & \infty & 6 & 0 \\ 0 & \infty & \infty & 0 \\ 2 & \infty & \infty & 0 \end{bmatrix} M_2$$

Cost reduced = 5

Cost of node 2 = Cost of node 1 + $M_1[a,b] + 2$

$$= 6 + 0 + 5 = 11$$

- Find matrix & cost of node 3.

- Set row a & column c elements are ∞

- Set $M_1[c,a] = \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 1 \\ \infty & 7 & \infty & 0 \\ 6 & 2 & \infty & \infty \end{bmatrix} \xRightarrow{RR} \begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 1 \\ \infty & 7 & \infty & 0 \\ 4 & 0 & \infty & \infty \end{bmatrix} \xRightarrow{CR} \begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & 1 \\ \infty & 7 & \infty & 0 \\ 4 & 0 & \infty & \infty \end{bmatrix} M_3$$

Cost reduced = 2

Cost (Node 3) = cost Node 1 + $m_1[a, c] + r$
 $= 6 + 3 + 2 = 11$

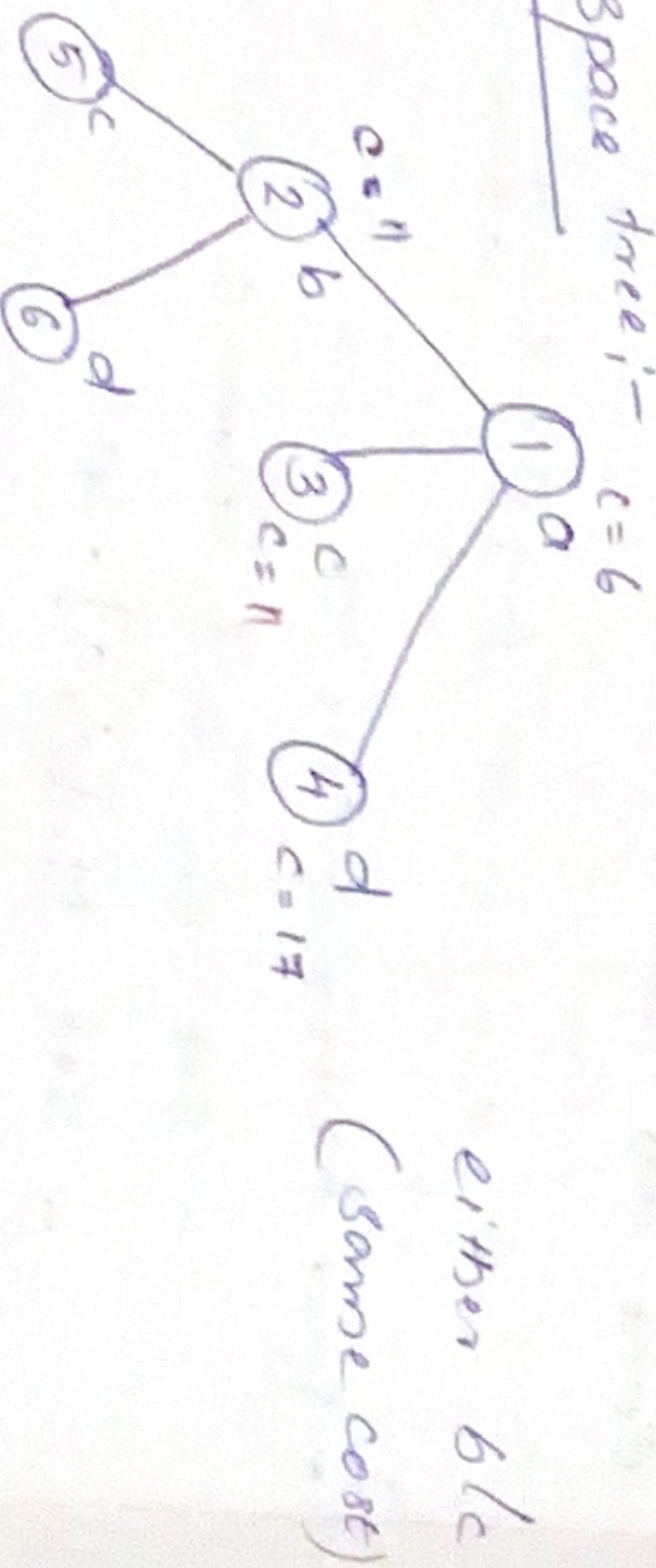
Node 4 : row 4 & col 1 $m_1[a, a] = \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & 6 & \infty \\ \infty & 7 & \infty & \infty \\ \infty & 2 & 0 & \infty \end{bmatrix} \xRightarrow{RR} \begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & 6 & \infty \\ 0 & 3 & \infty & \infty \\ \infty & 2 & 0 & \infty \end{bmatrix} \xRightarrow{C.P} \begin{bmatrix} \infty & \infty & \infty & \infty \\ 0 & \infty & 6 & \infty \\ 0 & 1 & \infty & \infty \\ \infty & 0 & 0 & \infty \end{bmatrix} M_4$$

Cost = 4 + 2 = 6

Cost Node 4 = 6 + 5 + 6 = 17

State space tree:-



Node 5 : row 6, column c, $m_2[c, c] = \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ 2 & \infty & \infty & \infty \end{bmatrix} \xRightarrow{RR} \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \end{bmatrix} \xRightarrow{CR} \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \end{bmatrix} M_5$$

Cost reduced = 2

Cost = Cost of Node 2 + $m_2[b, c] + r$
 $= 11 + 5 + 2 = 18$

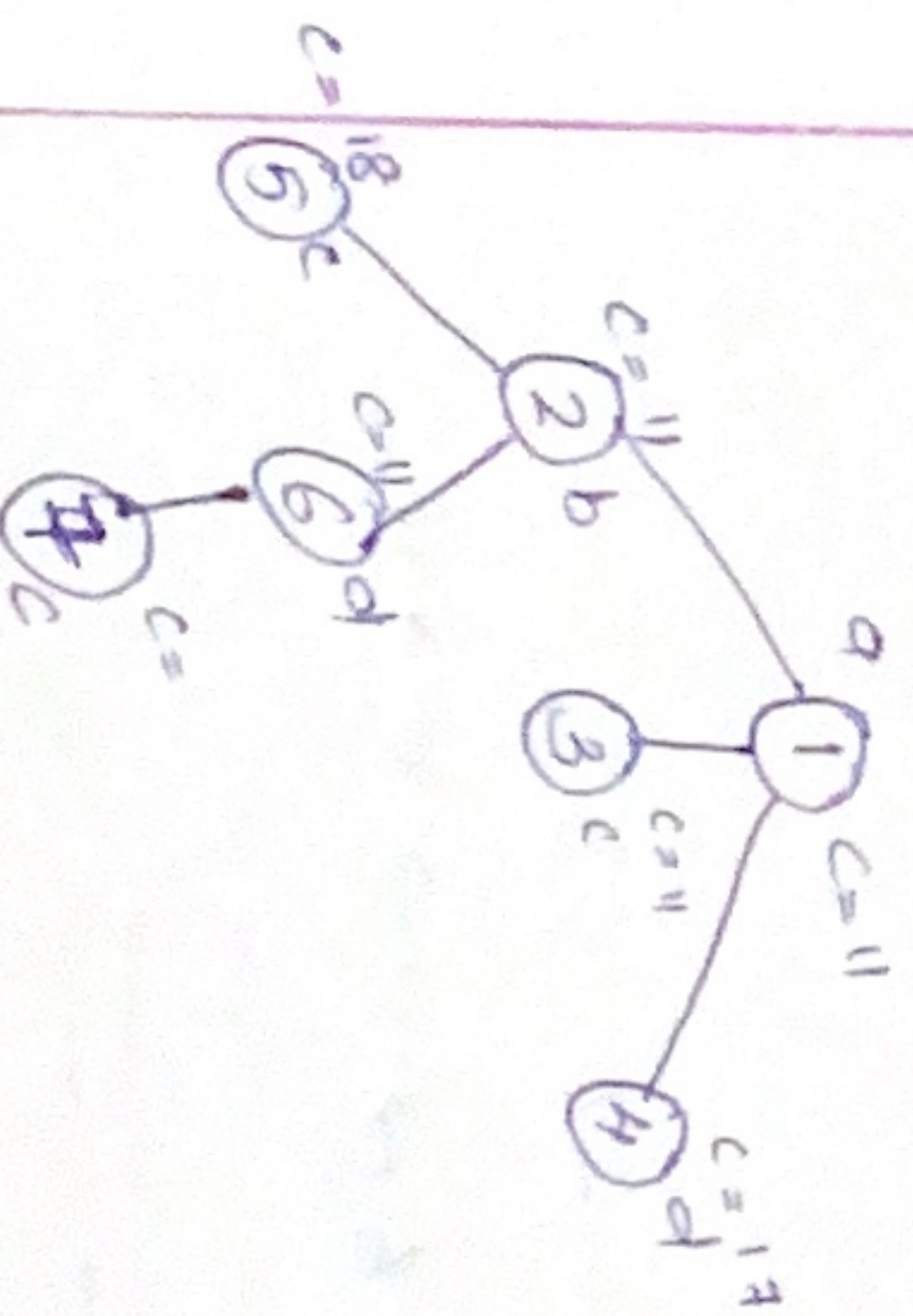
Node 6 : row 6, column d, $m_2[a, d] = \infty$

$$\begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \end{bmatrix} \xRightarrow{R.R} \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \end{bmatrix} \xRightarrow{C.R} \begin{bmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty \end{bmatrix} M_6$$

Cost reduced = 0

Cost Node 6 = Cost Node 2 + $m_2[b, d] + r$

$= 11 + 0 + 0 = 11$



Node f now d , c

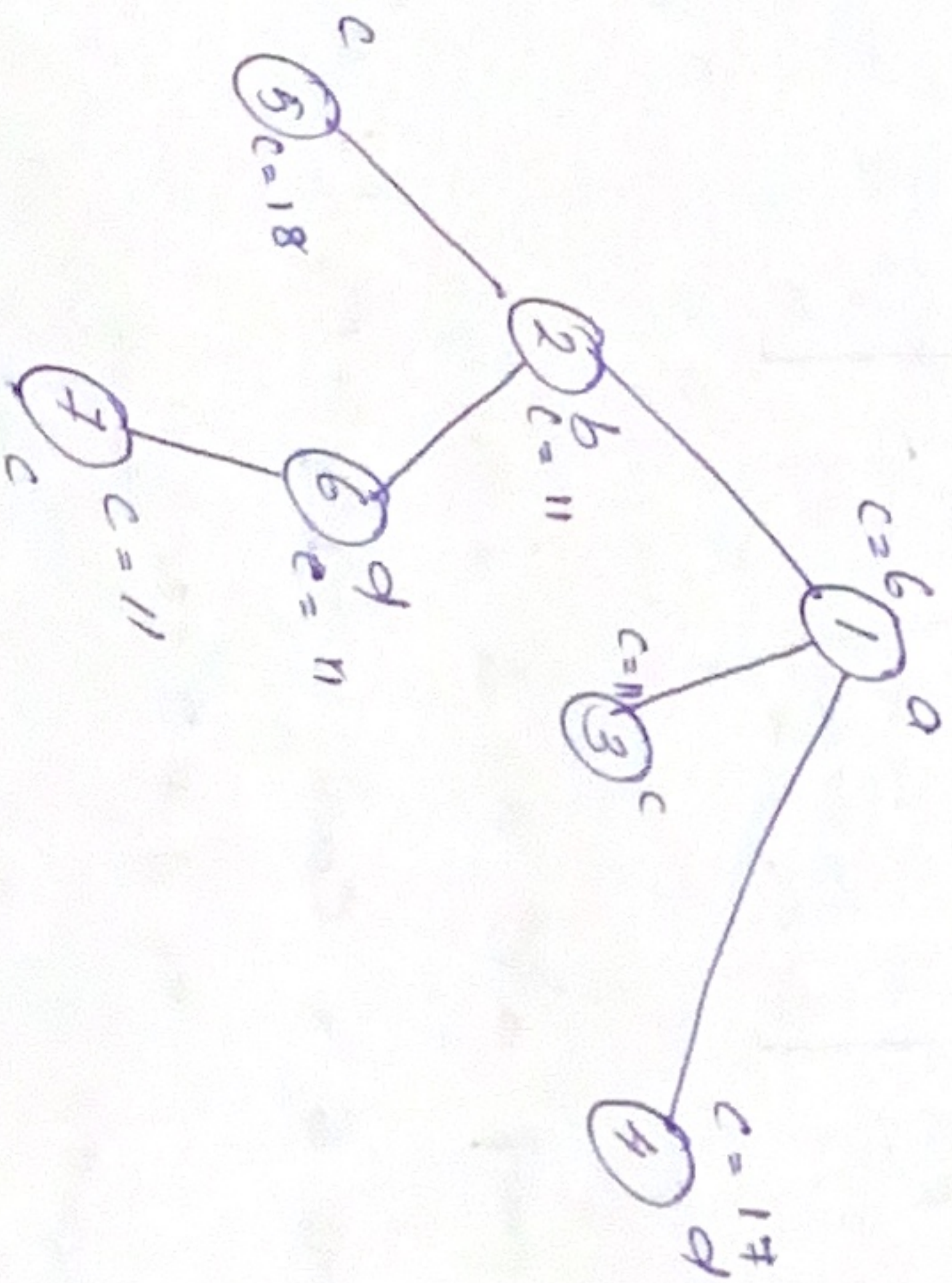
$$\begin{bmatrix} x & x & x & x \\ x & x & x & x \\ x & x & x & x \\ x & x & x & x \end{bmatrix} \xRightarrow{RR} \begin{bmatrix} x & x & x & x \\ x & x & x & x \\ x & x & x & x \\ x & x & x & x \end{bmatrix} \xRightarrow{C.R} \begin{bmatrix} x & x & x & x \\ x & x & x & x \\ x & x & x & x \\ x & x & x & x \end{bmatrix} M_f$$

Cost reduced = 0

$$\text{Cost node } f = \text{Cost node } c + M_b[a, c] + r$$

$$= 11 + 0 + 0 = \underline{\underline{11}}$$

State Space Tree



Cost of this level = 11

by path = $a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$